



CDA STUDIO · RÉVISION DU TITRE PROFESSIONNEL

Comprendre les notions. Puis les expliquer.

Un cours progressif, des exemples autonomes
et des exercices pour vérifier ce que tu as compris.

Merise · TypeScript · SQL · Docker
Tests JavaScript · Backend · React

Édition du 9 octobre 2026 · Exemples pédagogiques fictifs

Documentations officielles, spécifications et références académiques citées

À lire avec le recueil de cheatsheets pour le rappel rapide

Comment travailler avec ce PDF

Une notion à la fois : lis la définition, explique le cas fourni, puis tente l'exercice. Les corrections sont rassemblées après les chapitres pour pouvoir chercher avant de lire la réponse.

01 · Merise et conception / page 3

Comprendre les besoins, dessiner les données et construire un modèle relationnel - 8 leçons et 3 cheatsheets

02 · TypeScript / page 11

Comprendre les types, contrôler une entrée, structurer une fonction.

03 · SQL et PostgreSQL / page 14

Lire les lignes, comprendre leurs liens et modifier sans perdre la cohérence.

04 · Docker : exécuter et relier des applications / page 18

Comprendre les images, les ports et la conservation des données.

05 · Tests JavaScript : vérifier un comportement / page 21

Passer de la règle à un résultat attendu, puis tester les échanges.

06 · Backend : traiter une demande et protéger les données / page 24

HTTP, responsabilités, validation, permissions et accès à la base.

07 · React / page 28

Faire dépendre l'écran des données, puis synchroniser ce qui est extérieur au rendu.

Les trois supports ont des rôles différents

Cours : comprendre avec les explications et les exemples. Cheatsheets : retrouver rapidement une règle ou une syntaxe. Cartes du site : vérifier la mémoire sans regarder la réponse.

Les exemples ne demandent aucun souvenir de tes projets. Un extrait de code signale son contexte ; il n'est pas toujours une application complète à lancer. Le cours ne constitue pas un sujet officiel d'examen.

1. Partir d'un besoin vérifiable

Un besoin décrit le résultat utile attendu. Un acteur est un rôle dans l'utilisation du système, par exemple visiteur, lecteur ou administrateur.

La user story

Une user story exprime un besoin du point de vue d'un acteur : « En tant que [acteur], je souhaite [action], afin de [bénéfice]. » Elle ouvre une discussion ; elle ne fixe pas à elle seule toutes les règles.

Trois besoins d'une bibliothèque fictive

- En tant que visiteur, je souhaite rechercher un titre, afin de savoir si la bibliothèque possède le livre.
- En tant que lecteur inscrit, je souhaite consulter mes prêts actifs, afin de connaître les exemplaires à rendre.
- En tant qu'administrateur, je souhaite ajouter un exemplaire, afin de rendre une nouvelle copie disponible.

Critères d'acceptation : des résultats observables

Situation donnée	Résultat attendu
Titres : Le Phare et Les Jardins. Recherche : Phare.	Seul Le Phare apparaît.
Lina possède deux prêts actifs et un prêt rendu.	Sa liste contient les deux prêts actifs.
Le numéro d'inventaire 10 existe déjà.	Un nouvel exemplaire portant 10 est refusé.

De la règle aux données

Une règle de gestion impose une obligation métier : « un exemplaire appartient à un seul livre ». Le modèle de données représente les objets et liens nécessaires. Un rôle d'acteur ne devient pas automatiquement une entité ou une table.

ESSAIE SANS REGARDER LE CORRIGÉ · MER-01

Un prêt peut être renouvelé une seule fois, pour sept jours. Écris un critère pour un prêt déjà renouvelé.

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [01] Agile Alliance · Modèle de user story

[21] Pierre Gérard · Modélisation des systèmes d'information, cours universitaire

2. Définir les données et lire les cardinalités

Le dictionnaire des données précise le sens et les valeurs autorisées de chaque information. Le MCD, modèle conceptuel de données, représente les objets métier et leurs liens.

Le vocabulaire du MCD

Une entité est un type d'objet : LIVRE. Une occurrence est un objet précis : le livre 7. Un attribut décrit une propriété : titre. L'identifiant distingue chaque occurrence : livre_id. Une association exprime un lien : posséder. Une cardinalité indique le minimum et le maximum de participations d'une occurrence à ce lien.

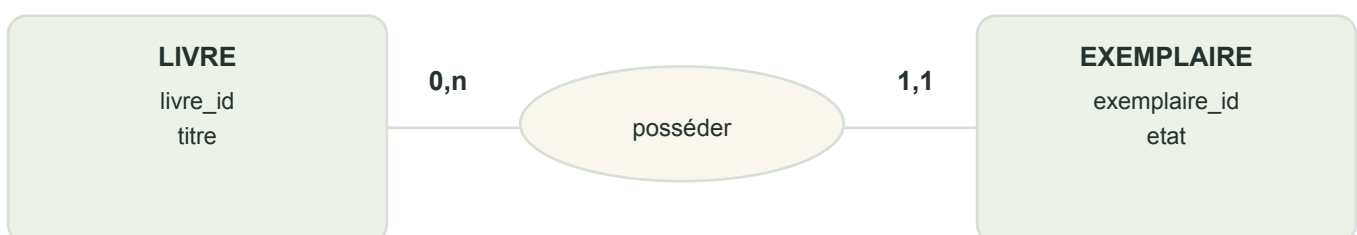
Petit dictionnaire

Donnée	Sens et domaine	Exemple
livre_id	Identifiant unique ; entier positif.	7
titre	Titre du livre ; texte non vide.	Le Phare
exemplaire_id	Identifiant unique d'une copie ; entier positif.	10

Les quatre cardinalités usuelles

Code	Minimum	Maximum	Lecture
0,1	0	1	Facultatif ; au plus un lien.
1,1	1	1	Obligatoire ; exactement un lien.
0,n	0	Plusieurs	Facultatif ; plusieurs liens possibles.
1,n	1	Plusieurs	Obligatoire ; au moins un lien.

Lire depuis chaque côté : un livre peut avoir zéro ou plusieurs copies ; chaque exemplaire appartient à exactement un livre



Un livre : de zéro à plusieurs exemplaires.

Un exemplaire : exactement un livre.

ESSAIE SANS REGARDER LE CORRIGÉ · MER-02

Le livre 7 existe sans copie. L'exemplaire 10 existe sans livre. Lequel respecte ce MCD ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [26] Nanci et Espinasse · Merise, deuxième génération (ouvrage des auteurs)

[18] Mocodo · Manuel de référence de l'auteur

3. Préserver les liens : binaire ou ternaire ?

Une association binaire relie deux types d'entités dans un fait. Une association ternaire en relie trois ensemble : un enseignant enseigne une matière à une classe.

Les seuls faits autorisés dans cet exemple

Enseignant	Matière	Classe
Alice	SQL	C1
Alice	JavaScript	C2
Bob	SQL	C2

Trois listes de paires peuvent produire une affectation inventée : Alice-SQL-C2

Faits autorisés

Alice - SQL - C1
Alice - JavaScript - C2
Bob - SQL - C2

Trois listes de paires

Rejoindre les trois listes
peut inventer Alice - SQL - C2

Les trois paires existent ; le triplet Alice - SQL - C2 n'existe pas.

Conserver une affectation complète, ou justifier la décomposition.

Pourquoi le faux triplet passe-t-il ?

Alice-SQL existe, Alice-C2 existe et SQL-C2 existe. Réunir ces trois paires semble donc autoriser Alice-SQL-C2. Pourtant, ce triplet n'est pas dans les faits. La projection en paires a perdu l'information sur leur combinaison exacte.

Choisir le bon modèle

- Binaire : LIVRE possède EXEMPLAIRE ; chaque fait relie un livre et une copie.
- Ternaire : AFFECTATION conserve enseignant, matière et classe dans le même fait.
- Décomposition sans perte : les relations séparées permettent de reconstruire exactement les faits, sans faux ajout. Elle doit être justifiée par les règles, pas par un dessin plus simple.

ESSAIE SANS REGARDER LE CORRIGÉ · MER-03

Les paires pourraient-elles autoriser Bob-JavaScript-C2 ? Indique la paire manquante.

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [26] Nanci et Espinasse · Merise, deuxième génération (ouvrage des auteurs)
[04] Sibel Adali, RPI · Dépendances, formes normales et décomposition

4. Du MCD au MLD : 1:n et n:n

Le MCD décrit les objets et les règles, sans choisir un moteur de base. Le MLD, modèle logique de données, les traduit ici en relations : ensembles de lignes et de colonnes.

Deux clés à comprendre

Une clé primaire, notée PK, identifie une ligne sans doublon ni valeur absente. Une clé étrangère, notée FK, référence une clé d'une autre relation. Elle garantit l'existence de la valeur référencée ; elle n'impose pas toute seule toutes les règles métier.

Conceptuel → logique → physique : trois niveaux de décision



Règles données et traduction

Lien	Règle de l'exemple	MLD
1:n	Chaque exemplaire appartient à un livre. Un livre peut avoir zéro ou plusieurs copies.	La FK livre_id va dans EXEMPLAIRE, le côté plusieurs.
n:n	Une commande contient un ou plusieurs produits. Un produit apparaît dans zéro ou plusieurs commandes. Au plus une ligne par couple.	Créer LIGNE ; sa clé est le couple commande-produit.
Attribut du lien	Chaque ligne possède une quantité positive.	quantite appartient à LIGNE, pas à PRODUIT.

MLD en notation abrégée - pas encore du SQL

```

LIVRE(livre_id PK, titre)
EXEMPLAIRE(exemplaire_id PK, livre_id FK → LIVRE)
COMMANDE(commande_id PK)
PRODUIT(produit_id PK)
LIGNE(commande_id FK → COMMANDE,
       produit_id FK → PRODUIT, quantite)
PK de LIGNE = (commande_id, produit_id)
  
```

ESSAIE SANS REGARDER LE CORRIGÉ · MER-04

Le livre 7 possède les copies 10 et 11. Quelles valeurs de livre_id trouve-t-on dans les deux lignes EXEMPLAIRE ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [18] Mocodo · Manuel de référence de l'auteur
[41] PostgreSQL · Contraintes

5. Liens 1:1, ternaire et MPD PostgreSQL

Le MPD, modèle physique de données, choisit les types et contraintes d'un moteur précis. Les exemples suivants utilisent PostgreSQL et des identifiants entiers.

Lire les contraintes avant le SQL

INTEGER stocke un entier. PRIMARY KEY identifie chaque ligne. NOT NULL rend une valeur obligatoire. UNIQUE interdit un doublon. REFERENCES impose que la valeur référencée existe. CHECK vérifie une condition, par exemple une quantité positive ; ajouter NOT NULL si la valeur doit aussi être présente.

Deux traductions particulières

Règle donnée	Traduction relationnelle
Une personne a zéro ou un dossier. Chaque dossier appartient à exactement une personne.	DOSSIER porte personne_id : FK, NOT NULL et UNIQUE.
Une affectation relie enseignant, matière et classe. Aucun triplet ne se répète ; aucune autre unicité n'est imposée.	AFFECTATION porte trois FK. La clé primaire contient les trois identifiants.

Cas 1:1 : deux personnes, un premier dossier

```
CREATE TABLE personne (
  id INTEGER PRIMARY KEY
);
CREATE TABLE dossier (
  id INTEGER PRIMARY KEY,
  personne_id INTEGER NOT NULL UNIQUE
  REFERENCES personne(id)
);
INSERT INTO personne VALUES (1), (2);
INSERT INTO dossier VALUES (10, 1);
```

Ce que le MPD doit encore préciser

- Choisir les types des autres attributs et les règles de suppression des références.
- Une FK ne garantit pas qu'un parent possède au moins un enfant : ce minimum demande un mécanisme supplémentaire.
- Un index aide certaines lectures ; ce choix physique ne remplace pas une règle du MCD.

ESSAIE SANS REGARDER LE CORRIGÉ · MER-05

Prédisez INSERT INTO dossier VALUES (11, 1), puis VALUES (12, 3), testés séparément après le code.

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [18] Mocodo · Manuel de référence de l'auteur
[41] PostgreSQL · Contraintes

6. Dépendances, clés et première forme normale

Une dépendance fonctionnelle $X \rightarrow Y$ signifie qu'un ensemble d'attributs X détermine Y : deux lignes ayant le même X doivent avoir le même Y . Cette règle vaut dans tout état valide, pas seulement dans un échantillon.

Les clés, sans raccourci

Une superclé identifie une ligne. Une clé candidate est une superclé minimale : aucun attribut n'est superflu. On choisit l'une des candidates comme clé primaire. Exemple donné : CLIENT(client_id, email, nom), avec client_id unique, email unique et obligatoire, et nom non unique.

Ce que les règles de CLIENT impliquent

Ensemble ou dépendance	Conclusion
{client_id} ; {email}	Deux clés candidates.
{client_id, nom}	Superclé, mais pas minimale.
client_id $\rightarrow$ email, nom	L'identifiant détermine les autres attributs.
nom $\rightarrow$ client_id	Faux : plusieurs clients peuvent avoir le même nom.

1NF : une valeur atomique par champ

La première forme normale, 1NF, exclut les groupes répétés et les collections dans une cellule du modèle relationnel classique. Le domaine choisi ici est un seul numéro de téléphone par champ. Pour deux numéros, créer deux lignes TELEPHONE(client_id, numero), plutôt qu'un champ téléphones contenant une liste.

Deux téléphones pour le client 1

```
CLIENT :
client_id | email | nom
1 | lina@example.test | Lina
TELEPHONE :
client_id | numero
1 | 0601
1 | 0602
```

ESSAIE SANS REGARDER LE CORRIGÉ · MER-06

Un échantillon présente trois noms différents. Peut-on en déduire que nom devient une clé candidate ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [37] Stéphane Crozat · Théorie de la normalisation relationnelle
[04] Sibel Adali, RPI · Dépendances, formes normales et décomposition

7. Normaliser : 2NF, 3NF et BCNF

Normaliser réduit les répétitions qui provoquent des anomalies d'insertion, de modification ou de suppression. Un attribut premier appartient à au moins une clé candidate ; un attribut non premier n'appartient à aucune.

Conditions à vérifier sur les dépendances

Forme	Condition
2NF	1NF ; aucun attribut non premier ne dépend d'une partie stricte d'une clé candidate.
3NF	Pour chaque $X \rightarrow A$ avec A hors de X : X est une superclé, ou A est premier.
BCNF	Pour chaque $X \rightarrow A$ avec A hors de X : X est une superclé. Aucune exception pour A premier.

2NF : retirer une dépendance partielle

LIGNE(commande, produit, libellé, quantité) a pour seule clé candidate (commande, produit). Règles : produit $\rightarrow$ libellé ; le couple $\rightarrow$ quantité. Le libellé dépend seulement d'une partie de la clé. Séparer PRODUIT(produit, libellé) et LIGNE(commande, produit, quantité).

3NF : retirer une dépendance transitive

COMMANDE(commande, client, nom_client) a pour seule clé candidate commande. Règles : commande $\rightarrow$ client ; client $\rightarrow$ nom_client. Séparer CLIENT(client, nom_client) et COMMANDE(commande, client). Un nom client n'est plus répété à chaque commande.

BCNF : une exigence plus forte

Autre cas : AFFECTATION(élève, matière, professeur). Règles : (élève, matière) $\rightarrow$ professeur ; professeur $\rightarrow$ matière. Les candidates sont (élève, matière) et (élève, professeur). La relation respecte 3NF : matière est première. Elle viole BCNF : professeur n'est pas une superclé. Une décomposition doit vérifier reconstruction sans perte et conservation des dépendances.

ESSAIE SANS REGARDER LE CORRIGÉ · MER-07

Ajouter seulement un id artificiel à LIGNE corrige-t-il sa dépendance produit $\rightarrow$ libellé ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [37] Stéphane Crozat · Théorie de la normalisation relationnelle
[04] Sibel Adali, RPI · Dépendances, formes normales et décomposition

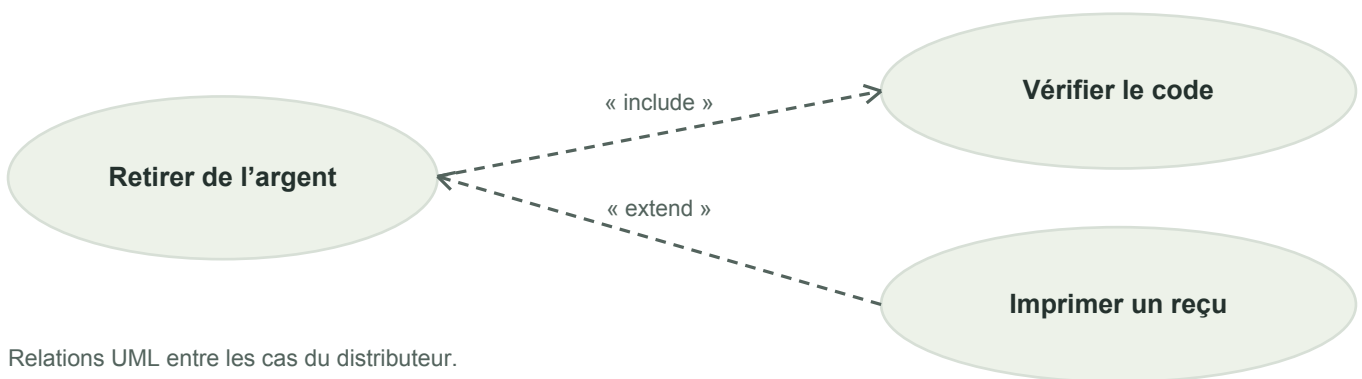
8. Cas d'utilisation UML : include et extend

UML est un langage de modélisation défini par l'OMG. Un cas d'utilisation décrit un service utile à un acteur ; un MCD Merise décrit les données et leurs relations.

Le système et son scénario

Système : distributeur bancaire. Acteur extérieur : client. Cas : Retirer de l'argent. Conditions de notre exemple : carte valide et solde suffisant. Le retrait vérifie toujours le code. Si le code est faux, aucun argent n'est délivré. Un reçu est proposé à un point du retrait et imprimé seulement sur demande ; le retrait reste complet sans reçu.

Retirer inclut Vérifier le code ; Imprimer le reçu étend Retirer



Relations UML entre les cas du distributeur.

Deux relations, deux directions

Relation	Sens de la flèche	Signification ici
include	Retirer → Vérifier le code	Le comportement inclus est exécuté dans le retrait.
extend	Imprimer le reçu → Retirer	Extension conditionnelle à un point prévu du retrait.

Décrire et vérifier le cas

- Délimiter le système ; nommer les acteurs et les objectifs, pas seulement les écrans.
- Écrire préconditions, étapes, alternatives et résultat : code faux → retrait refusé.
- Le cas étendu doit être complet sans son extension. La flèche de dépendance n'est pas une flèche de chronologie.
- L'acteur Client n'impose pas automatiquement une nouvelle entité dans le MCD.

ESSAIE SANS REGARDER LE CORRIGÉ · MER-08

Nouvelle règle : un reçu est imprimé à chaque retrait. Quelle relation utiliser et dans quel sens ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [40] OMG · UML 2.5.1, standard et document normatif, chapitre 18

1 · Des types pour relire le code avant son exécution

TypeScript ajoute un contrôle statique : il vérifie le code avant son exécution. Ses types sont ensuite effacés ; une donnée reçue doit encore être contrôlée pendant l'exécution.

Nommer la forme d'une valeur

Forme	Signification
number, string, boolean	Nombre, texte, vrai/faux.
string null	Texte ou absence : union de possibilités.
interface Livre { titre: string }	Forme d'un objet nommée Livre.
type Code = string number	Nom réutilisable pour une union.
string[] ; [string, number]	Tableau de textes ; tuple à deux positions typées.
readonly string[]	Interdit la modification via cette référence ; pas un gel à l'exécution.

Programme TypeScript complet : exemple.ts

```
interface Livre { titre: string; pages?: number }
type Position = [string, number];
const livre: Livre = { titre: 'Lire' };
const position: Position = ['rayon', 2];
const tags: readonly string[] = ['lecture'];
console.log(livre.titre, position[1], tags[0]);
// Attendu : Lire 2 lecture
```

Outils et contrat de lecture

- Dans un dossier d'essai : `npm init -y`, puis `npm install -D typescript tsx`.
- `npx tsc --strict --noEmit exemple.ts` vérifie sans produire de JavaScript. `strict` renforce notamment les contrôles d'absences et de `any` implicites.
- `npx tsx exemple.ts` exécute le programme ; `tsx` ne remplace pas la vérification `tsc`.
- `pages?` est facultatif. Une interface décrit surtout un objet ; `type` peut aussi nommer union ou tuple. L'inférence déduit des types depuis les valeurs.

ESSAIE SANS REGARDER LE CORRIGÉ · TYP-01

Que décrit `[string, number]` ? Le contrôle TypeScript prouve-t-il qu'un JSON reçu respecte Livre ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [\[62\]](#) [\[63\]](#) [\[68\]](#) [\[70\]](#) [\[38\]](#) · Notices complètes à la fin ; liens cliquables.

2 - Accepter une valeur inconnue, puis vérifier

Une entrée extérieure peut avoir n'importe quelle forme. `unknown` oblige à la vérifier ; une branche de code peut ensuite réduire les types possibles : c'est le `narrowing`.

Quatre notions à distinguer

Notion	Ce qu'elle fait
<code>unknown</code>	Valeur inconnue ; vérifier avant une opération précise.
<code>any</code>	Désactive des contrôles utiles ; les erreurs restent possibles.
<code>value as Livre</code>	Affirme un type au compilateur ; ne transforme ni ne valide.
<code>typeof</code> / <code>instanceof</code> / test d'un champ	Réduit les possibilités dans la branche vérifiée.
Union discriminée	Variante distinguées par un champ commun, comme <code>status</code> .

Programme complet : deux résultats possibles

```
type Result =
  | { status: 'ok'; data: string }
  | { status: 'error'; message: string };
function read(value: unknown): Result {
  if (typeof value === 'string') {
    return { status: 'ok', data: value.trim() };
  }
  return { status: 'error', message: 'Texte requis' };
}
const result = read(12);
if (result.status === 'ok') console.log(result.data);
else console.log(result.message);
// Attendu : Texte requis
```

Passer d'une preuve à l'autre

- `typeof value === 'string'` permet `trim` dans cette branche : la valeur est un texte.
- Tester `status` donne ensuite accès à `data` OU `message` selon la variante.
- Pour un objet reçu en JSON : vérifier objet non nul, champs attendus, types et règles métier. Une quantité positive exige plus qu'un simple type `number`.

ESSAIE SANS REGARDER LE CORRIGÉ · TYP-02

Avec `read(' Alice ')`, quel résultat obtient-on ? Pourquoi `result.message` est-il incorrect dans la branche `status === 'ok'` ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [67] TypeScript · Narrowing et unions discriminées
[64] TypeScript · Fonctions
[63] TypeScript · Types courants

3 · Réutiliser un type et attendre un calcul

Un générique conserve une relation entre types ; une fonction async retourne une Promise, représentant un résultat futur. Un module organise les exports et imports entre fichiers.

Types construits depuis User

Outil	Résultat
Partial<User>	Toutes les propriétés deviennent facultatives.
Pick<User, 'name'>	Conserve uniquement name.
Omit<User, 'id'>	Retire id.
Readonly<User>	Interdit la réaffectation des propriétés par ce type.
Record<string, number>	Associe des clés textuelles à des nombres.

Programme en deux fichiers d'un dossier d'essai

```
// users.ts
export type User = { id: number; name: string };
export function first<T>(values: readonly T[]): T | undefined {
  return values[0];
}
export async function count(users: readonly User[]): Promise<number> {
  return users.length;
}
// main.ts
import { first, count } from './users.js';
import type { User } from './users.js';
const users: User[] = [{ id: 1, name: 'Alice' }];
async function main() {
  console.log(first(users)?.name, await count(users));
}
main(); // Attendu : Alice 1
```

Lire et exécuter

- T représente le type des éléments ; first garde ce type et prévoit undefined pour une liste vide.
- await lit le résultat de la Promise dans une fonction async ; un rejet peut être traité avec try/catch. Vérifier error instanceof Error avant error.message si l'erreur est unknown.
- import type importe une définition effacée à l'exécution. Le chemin users.js désigne le fichier JavaScript produit.
- Après installation de TypeScript : npx tsc --strict --target ES2022 --module NodeNext --outDir dist main.ts, puis node dist/main.js.

ESSAIE SANS REGARDER LE CORRIGÉ · TYP-03

Que retourne first<number>([]) ? Quel est le type de count(users) avant await ? Partial<User> valide-t-il un objet réseau ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [65] [69] [66] [64] [67] · Notices complètes à la fin ; liens cliquables.

1 · Tables, identifiants et intégrité

Une table possède des colonnes et des lignes. Notre jeu contient Alice avec deux commandes et Bilal avec aucune ; chaque commande doit référencer un client existant.

Définir les règles avant les colonnes

- PRIMARY KEY identifie une ligne sans doublon ni NULL. FOREIGN KEY impose une référence existante.
- NOT NULL impose une valeur ; UNIQUE interdit un doublon selon sa règle ; CHECK vérifie une condition.
- CHECK(total >= 0) seul laisse passer NULL dans PostgreSQL : ajouter NOT NULL rend le total obligatoire.
- Programme SQL autonome ci-dessous : exécuter dans une seule connexion PostgreSQL de test. TEMP crée des tables limitées à cette session.

Créer notre petit jeu de données

```
CREATE TEMP TABLE client (  
  id INTEGER PRIMARY KEY, nom TEXT NOT NULL  
);  
CREATE TEMP TABLE commande (  
  id INTEGER PRIMARY KEY,  
  client_id INTEGER NOT NULL REFERENCES client(id),  
  total NUMERIC NOT NULL CHECK (total >= 0)  
);  
INSERT INTO client VALUES (1, 'Alice'), (2, 'Bilal');  
INSERT INTO commande VALUES (10, 1, 30), (11, 1, 20);
```

Données obtenues

Table	Lignes
client	(1, Alice) ; (2, Bilal)
commande	(10, client 1, 30) ; (11, client 1, 20)

Ce qui est attendu

Deux clients et deux commandes existent. Les deux références pointent vers Alice. Les colonnes décrivent les données ; les contraintes empêchent certains états interdits, même en dehors d'un formulaire.

ESSAIE SANS REGARDER LE CORRIGÉ · SQL-01

Peut-on ajouter (12, 3, 15) dans commande ? Peut-on ajouter (12, 2, -1) ? Quelle règle bloque chaque tentative ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [41] PostgreSQL · Contraintes
[53] PostgreSQL · INSERT et données d'exemple

2 · Lire, filtrer, trier et découper

SELECT décrit les valeurs à obtenir, FROM les fournit et WHERE filtre les lignes. Le jeu est redéclaré dans la requête : aucune table personnelle n'est nécessaire.

Le chemin de lecture

- SELECT choisit les colonnes ; WHERE conserve les lignes dont la condition vaut vrai.
- ORDER BY total DESC trie du plus grand au plus petit ; id départage un éventuel ex æquo.
- LIMIT conserve un nombre maximal de lignes ; OFFSET ignore les premières lignes du résultat trié.
- WITH nom(...) AS (VALUES ...) définit ici des données temporaires valables uniquement pour cette requête.

Requête PostgreSQL autonome

```
WITH client(id, nom) AS (VALUES (1, 'Alice'), (2, 'Bilal')),
commande(id, client_id, total) AS (
  VALUES (10, 1, 30), (11, 1, 20)
)
SELECT id, total
FROM commande
WHERE total >= 20
ORDER BY total DESC, id
LIMIT 1 OFFSET 1;
```

Résultat attendu

id	total
11	20

Pourquoi ce résultat ?

Le filtre garde 30 et 20. Le tri place 30 avant 20. OFFSET 1 ignore 30, puis LIMIT 1 conserve 20. Sans ordre défini, une pagination ne garantit pas une sélection stable.

ESSAIE SANS REGARDER LE CORRIGÉ · SQL-02

Avec les mêmes données, remplace WHERE par total >= 25 et supprime LIMIT/OFFSET. Quelle ligne reste ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [54] PostgreSQL · Interroger une table

[48] PostgreSQL · ORDER BY

[47] PostgreSQL · LIMIT et OFFSET

3 · Joindre, comprendre NULL et compter

Une jointure rapproche les lignes de tables liées. LEFT JOIN garde aussi un client sans commande ; les colonnes de la commande valent alors NULL, l'absence de valeur.

Éviter les pièges

- INNER JOIN garde seulement les correspondances ; LEFT JOIN garde aussi les lignes de gauche sans correspondance.
- COUNT(co.id) compte les identifiants non NULL : Alice 2, Bilal 0. COUNT(*) compterait aussi la ligne conservée pour Bilal.
- GROUP BY forme les groupes ; WHERE filtre avant, HAVING après le regroupement.
- NULL se teste avec IS NULL, jamais = NULL. COALESCE prend la première valeur non NULL ; ici il remplace une somme absente par 0.

Requête autonome : une ligne par client

```
WITH client(id, nom) AS (VALUES (1, 'Alice'), (2, 'Bilal')),
commande(id, client_id, total) AS (
  VALUES (10, 1, 30), (11, 1, 20)
)
SELECT c.nom, COUNT(co.id) AS commandes,
       COALESCE(SUM(co.total), 0) AS total
FROM client c
LEFT JOIN commande co ON co.client_id = c.id
GROUP BY c.id, c.nom
ORDER BY c.id;
```

Résultat attendu

nom	commandes	total
Alice	2	50
Bilal	0	0

Lire la ligne vide

La jointure fabrique deux correspondances pour Alice et conserve une ligne sans commande pour Bilal. Cette absence n'est pas un total nul : SUM ne trouve aucune valeur, puis COALESCE fournit 0.

ESSAIE SANS REGARDER LE CORRIGÉ · SQL-03

Ajoute HAVING COUNT(co.id) >= 2 avant ORDER BY. Qui reste ? Pour Bilal, que donnerait COUNT(*) sans cette condition ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [52] [43] [50] [44] · Notices complètes à la fin ; liens cliquables.

4 · Existence, modifications et travail fiable

EXISTS teste si une sous-requête trouve au moins une ligne. Nous gardons clients (1, Alice), (2, Bilal) et commandes (10, 1, 30), (11, 1, 20).

Lecture autonome : clients ayant une commande

```
WITH client(id, nom) AS (VALUES (1, 'Alice'), (2, 'Bilal')),
commande(id, client_id, total) AS (
  VALUES (10, 1, 30), (11, 1, 20)
)
SELECT c.nom FROM client c
WHERE EXISTS (
  SELECT 1 FROM commande co WHERE co.client_id = c.id
);
-- Attendu : Alice, une seule fois.
```

Modifier et vérifier

Outil	Sens
INSERT ... RETURNING id	Ajouter et récupérer l'identifiant de sa ligne.
UPDATE / DELETE + WHERE	Modifier / supprimer les lignes ciblées.
Paramètres (\$1, valeurs séparées)	Séparer une valeur du code SQL ; pas un nom de colonne.
BEGIN / COMMIT / ROLLBACK	Débuter / valider / annuler une transaction.
Index	Accélérer certains accès, avec coût d'espace et d'écriture.
EXPLAIN / EXPLAIN ANALYZE	Plan estimé / exécution mesurée, donc effets réels possibles.

Fragment à lire : tables et données décrites ci-dessus

```
BEGIN;
UPDATE commande SET total = 99 WHERE id = 10;
ROLLBACK;
-- Après annulation : commande 10 vaut toujours 30.
-- Pilote applicatif : SQL '... WHERE id = $1', valeurs [10].
-- Un nom de colonne dynamique vient d'une liste autorisée.
```

Relier l'outil au besoin

Une transaction regroupe des changements qui doivent être validés ensemble. Avant UPDATE/DELETE, vérifier les lignes visées. Ajouter un index selon les requêtes observées ; un index ne rend pas forcément chaque requête plus rapide.

ESSAIE SANS REGARDER LE CORRIGÉ · SQL-04

Après le fragment, quelle valeur conserve la commande 10 ? Un UPDATE sans WHERE viserait combien de commandes ici ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [\[45\]](#) [\[49\]](#) [\[55\]](#) [\[42\]](#) [\[56\]](#) [\[51\]](#) [\[46\]](#) [\[57\]](#) · Notices complètes à la fin ; liens cliquables.

De l'image au conteneur

Une image est un paquet de fichiers et de réglages. Un conteneur est une exécution isolée de cette image ; un Dockerfile décrit sa fabrication.

Lire les instructions

Instruction	Rôle
FROM	Choisir l'image de base.
WORKDIR	Fixer le dossier de travail.
COPY	Copier des fichiers dans l'image.
RUN	Exécuter pendant la fabrication.
CMD	Définir la commande de démarrage.
EXPOSE	Documenter un port ; ne pas le publier.

Deux fichiers et des commandes : séparer les trois parties

```
# api.mjs : exclure cette ligne de titre
import { createServer } from "node:http";
createServer((req, res) => {
  res.setHeader("Content-Type", "application/json");
  res.end('{"ok":true}');
}).listen(3000, "0.0.0.0");
# Dockerfile : exclure cette ligne de titre
FROM node:22-alpine
WORKDIR /app
COPY api.mjs .
RUN node --check api.mjs
CMD ["node", "api.mjs"]
# Terminal, depuis le dossier des deux fichiers
docker build -t atelier-api .
docker run --name api -p 127.0.0.1:8080:3000 atelier-api
```

Pratiquer et vérifier

Prérequis : Docker installé et démarré, Internet pour l'image de base, port 8080 libre. Créer api.mjs et Dockerfile dans un dossier vide. build fabrique l'image ; run crée et démarre le conteneur. Ouvrir <http://127.0.0.1:8080> : réponse attendue {"ok":true}. Un port déjà occupé fait échouer run.

Ce qui mérite d'être retenu

- RUN vérifie ici la syntaxe du fichier ; CMD démarre le serveur plus tard.
- Le cache réutilise des étapes inchangées. Copier les fichiers de dépendances avant les sources limite les réinstallations.
- .dockerignore exclut des fichiers du build. Ne pas copier de secret dans l'image.

ESSAIE SANS REGARDER LE CORRIGÉ · DOC-01

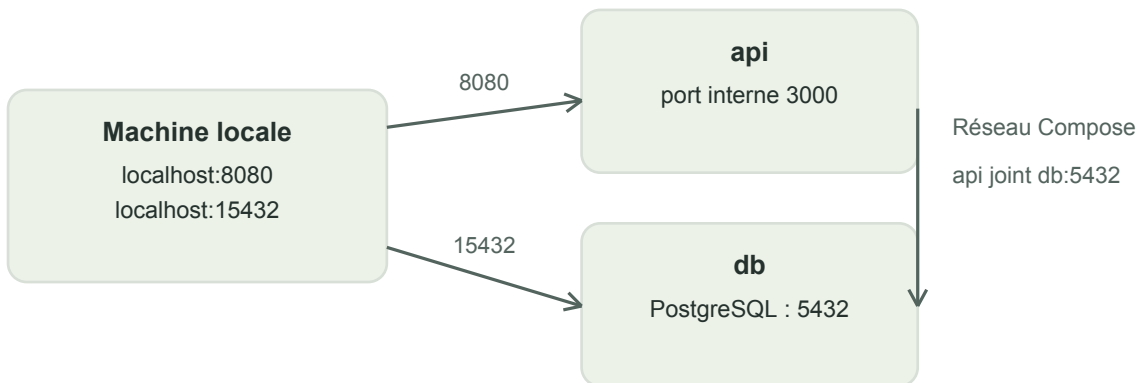
Ajouter EXPOSE 3000 au Dockerfile rend-il le serveur accessible sur le port 8080 de la machine ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [11] [10] [13] [09] · Notices complètes à la fin ; liens cliquables.

Ports, réseau Compose et données persistantes (1/2)

Compose décrit plusieurs services et leur réseau. Un volume nommé conserve des fichiers indépendamment du conteneur qui les a écrits.



Choisir l'adresse depuis le bon endroit

Programme appelant	Destination
Navigateur sur la machine	API : localhost:8080
API dans Compose	PostgreSQL : db:5432
Outil SQL sur la machine	PostgreSQL : localhost:15432
localhost dans api	Le conteneur api lui-même

RÉFÉRENCES · [06] [08] [07] [12] · Notices complètes à la fin ; liens cliquables.

Ports, réseau Compose et données persistantes (2/2)

Compose décrit plusieurs services et leur réseau. Un volume nommé conserve des fichiers indépendamment du conteneur qui les a écrits.

compose.yaml : lecture du réseau et du stockage

```
services:
  api:
    image: atelier-api
    ports: ["127.0.0.1:8080:3000"]
  db:
    image: postgres:17
    environment:
      POSTGRES_PASSWORD: exemple-local
    ports: ["127.0.0.1:15432:5432"]
    volumes: ["records:/var/lib/postgresql/data"]
volumes:
  records:
```

Lancer, diagnostiquer, conserver

- atelier-api est l'image de la page précédente. Cette API répond en JSON mais ne lit pas encore la base ; db:5432 est l'adresse qu'elle utiliserait.
- Arrêter le conteneur précédent avec Ctrl+C dans son terminal ou docker stop api depuis un autre terminal : cela libère 8080. Depuis le dossier de compose.yaml : docker compose up -d, puis docker compose ps et docker compose logs api db. La base accepte postgres avec le mot de passe d'exemple.
- depends_on simple attend le démarrage, pas la disponibilité. service_healthy attend un healthcheck initial ; l'application gère ensuite les pannes.
- records conserve les fichiers de PostgreSQL. down conserve les volumes nommés ; down -v les supprime. Un volume ne remplace pas une sauvegarde.

ESSAIE SANS REGARDER LE CORRIGÉ · DOC-02

Retirer la publication 15432:5432 empêche-t-il api de joindre db ? Et l'outil SQL sur la machine ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [06] [08] [07] [12] · Notices complètes à la fin ; liens cliquables.

Une règle, des limites et une assertion (1/2)

Un test automatisé exécute une action puis vérifie son résultat. Une assertion est la vérification ; le test runner organise l'exécution des tests.

Unitaire

Une règle
Dépendances remplacées

Intégration

Composants réunis
Base ou HTTP réels

Parcours

Action utilisateur
Résultat visible

Règle : accepter exactement les entiers de 1 à 5

Entrée	Résultat attendu
0	false : sous la limite
1	true : borne incluse
3	true : intérieur
5	true : borne incluse
6	false : au-dessus
"3", 1.5 ou NaN	false : pas un entier admis

RÉFÉRENCES · [25] Node.js · exécuter des tests

[23] Node.js · vérifier un résultat avec assert

[20] Microsoft Learn · organiser des tests unitaires avec Arrange, Act, Assert

Une règle, des limites et une assertion (2/2)

Un test automatisé exécute une action puis vérifie son résultat. Une assertion est la vérification ; le test runner organise l'exécution des tests.

Node 22+ : `quantite.test.mjs`, puis `node --test quantite.test.mjs`

```
import test from "node:test";
import assert from "node:assert/strict";
const acceptQuantity = value =>
  Number.isInteger(value) && value >= 1 && value <= 5;
test("refuse zéro", () => {
  const value = 0; // Arrange : préparer
  const actual = acceptQuantity(value); // Act : agir
  assert.strictEqual(actual, false); // Assert : vérifier
});
```

Comprendre ce que le test prouve

- Résultat attendu : un test réussit. Ajouter les cas du tableau, chacun avec son attendu ; un seul cas ne prouve pas toutes les limites.
- Unitaire : une unité de logique. Intégration : coopération entre composants. E2E : parcours utilisateur complet.
- `strictEqual` compare ici un booléen ; `deepStrictEqual` compare la structure d'un tableau ou objet.
- Les données préparées sont des fixtures. Un double remplace une dépendance pour contrôler son résultat ; il ne vérifie pas cette dépendance réelle.

ESSAIE SANS REGARDER LE CORRIGÉ · TES-01

La règle devient « entiers de 2 à 8 inclus ». Quelles quatre valeurs prennent les bornes et leurs voisins extérieurs ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [25] Node.js · exécuter des tests

[23] Node.js · vérifier un résultat avec `assert`

[20] Microsoft Learn · organiser des tests unitaires avec `Arrange`, `Act`, `Assert`

Attendre, tester HTTP et nettoyer

Une promesse représente un résultat futur ; `await` l'attend. Ce test d'intégration vérifie le statut et le corps d'une réponse HTTP réelle.

Node 22+ : `http.test.mjs`, puis `node --test http.test.mjs`

```
import test from "node:test";
import assert from "node:assert/strict";
import { createServer } from "node:http";
import { once } from "node:events";
test("GET /tasks/1", async t => {
  const server = createServer((req, res) => {
    const found = req.method === "GET" && req.url === "/tasks/1";
    res.writeHead(found ? 200 : 404, {"Content-Type":"application/json"});
    res.end(JSON.stringify(found ? {id:1,title:"Lire"} : {error:"Absente"}));
  });
  t.after(() => new Promise(resolve => server.close(resolve)));
  server.listen(0, "127.0.0.1"); await once(server, "listening");
  const url = "http://127.0.0.1:" + server.address().port;
  const response = await fetch(url + "/tasks/1");
  assert.strictEqual(response.status, 200);
  assert.deepStrictEqual(await response.json(), {id:1,title:"Lire"});
});
```

Lire le scénario

- Fixture fournie : `GET /tasks/1` → 200 et `{id:1,title:"Lire"}` ; autre route → 404 et `{error:"Absente"}`.
- `listen(0)` choisit un port libre ; `listening` attend le démarrage. `t.after` ferme le serveur après le test.
- `fetch` ne rejette pas automatiquement sur 404 : vérifier statut et corps, avec `await`.
- Intégration avec une base : données connues réinitialisées, base dédiée et connexions libérées.

Deux outils, mêmes intentions

Vérification	Node assert	Vitest expect
Valeur simple	<code>strictEqual(a, b)</code>	<code>toBe(b)</code>
Structure	<code>deepStrictEqual(a, b)</code>	<code>toEqual(b)</code>
Promesse rejetée	<code>await rejects(fn, motif)</code>	<code>await expect(p).rejects.toThrow(motif)</code>

TDD : construire avec le test

TDD : Red, test en échec ; Green, code qui le fait passer ; Refactor, amélioration sans changer le comportement. Vitest lance une suite équipée avec `npx vitest run`.

ESSAIE SANS REGARDER LE CORRIGÉ · TES-02

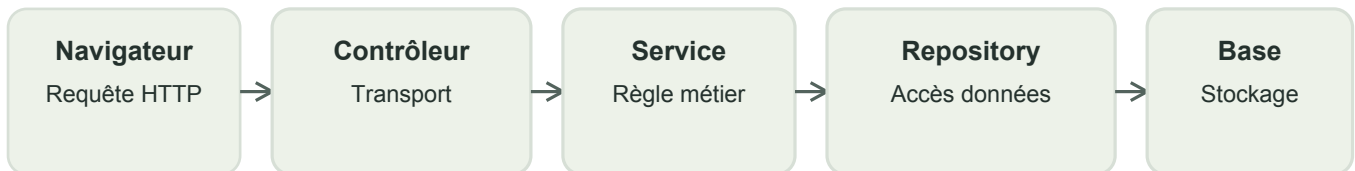
`GET /tasks/99` : quels deux attendus vérifier ? Quel niveau de test est-ce ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [25] [23] [24] [39] [19] · Notices complètes à la fin ; liens cliquables.

La requête traverse des responsabilités (1/2)

Une API est un point d'accès offert par un serveur ; une route associe une méthode HTTP à un chemin. Node exécute JavaScript et Express organise le traitement des requêtes.



La réponse revient vers le navigateur : statut HTTP et données utiles.

Avant le code

- Exemple autonome : Node 22+, Express 5 installé avec `npm install express@5` ; enregistrer `api.mjs`, puis `node api.mjs`.
- Middleware : étape qui répond ou appelle `next()`. Décoder le JSON avant les routes ; terminer par le traitement des erreurs.
- Contrôleur : adapter `req` et `res` ; service : appliquer ou orchestrer les règles ; repository : lire et écrire les données. Ici, le repository utilise un tableau.

RÉFÉRENCES · [16] [15] [14] [61] · Notices complètes à la fin ; liens cliquables.

La requête traverse des responsabilités (2/2)

Une API est un point d'accès offert par un serveur ; une route associe une méthode HTTP à un chemin. Node exécute JavaScript et Express organise le traitement des requêtes.

api.mjs : une tâche et trois réponses possibles

```
import express from "express";
const app = express();
app.use(express.json());
const tasks = [{id:1, title:"Lire"}];
const repo = { find: id => tasks.find(t => t.id === id) };
const service = id => repo.find(id);
app.get("/tasks/:id", (req, res) => {
  const id = Number(req.params.id);
  if (!Number.isInteger(id) || id < 1)
    return res.status(400).json({error:"Id invalide"});
  const task = service(id);
  return task ? res.json(task) : res.status(404).json({error:"Absente"});
});
app.use((err, req, res, next) =>
  res.status(err.status || 500).json({error:"Incident"}));
app.listen(3000, "127.0.0.1");
```

Depuis http://127.0.0.1:3000

Requête	Résultat attendu
GET /tasks/1	200 ; {id:1,title:"Lire"}
GET /tasks/99	404 ; {error:"Absente"}
GET /tasks/abc	400 ; {error:"Id invalide"}
Exception transmise	500 ; message générique, sans détails internes
GET /tasks/1 avec Content-Type: application/json et corps {	400 ; statut de l'erreur JSON préservé

ESSAIE SANS REGARDER LE CORRIGÉ · BAC-01

req.params.id vaut "1". Qui adapte ce texte et qui recherche la tâche ? Que se passe-t-il sans réponse ni next() ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [16] [15] [14] [61] · Notices complètes à la fin ; liens cliquables.

Valider les données, vérifier les droits

Zod est une bibliothèque de validation à l'exécution. TypeScript contrôle statiquement le code ; authentifier établit une identité, autoriser décide de ses droits.

Zod 4 : fragment à lire, trois corps JSON fournis

```
import { z } from "zod";
const booking = z.object({
  seats: z.number().int().min(1).max(4)
});
for (const body of [{seats:2}, {seats:"2"}, {seats:5}]) {
  console.log(booking.safeParse(body).success);
} // Résultats : true, false, false
```

Contrat de cette API de réservation

Situation	Réponse
Création valide effectuée	201 : ressource créée
JSON incompréhensible	400 : requête incorrecte
JSON compris, seats invalide	422 : validation refusée
Identité absente ou invalide	401 + défi WWW-Authenticate adapté
Identité valide, permission absente	403 : action refusée

JWT et mot de passe : deux mécanismes

Le contenu d'un JWT signé est lisible. verify vérifie la signature et exp (date d'expiration) lorsqu'il est présent. Configurer algorithms (algorithmes autorisés), issuer (émetteur) et audience (destinataire) pour imposer les valeurs attendues ; decode lit sans vérifier. Contrôler les droits par ressource. Un sel est une valeur aléatoire propre à chaque mot de passe : utiliser un hachage adapté, comme Argon2id, jamais le texte clair.

Sécurité du navigateur et du serveur

- Règle seats : entier de 1 à 4. safeParse rend success avec data ou une erreur ; utiliser data après succès.
- HttpOnly interdit la lecture JavaScript du cookie ; Secure réserve son envoi à HTTPS.
- CSRF : requête forgée exploitant un cookie envoyé automatiquement. Protéger les actions sensibles ; SameSite complète ces protections.
- Lire le propriétaire enregistré côté serveur. Garder secrets et données sensibles hors des logs.

ESSAIE SANS REGARDER LE CORRIGÉ · BAC-02

Le document 42 appartient à 9. L'utilisateur authentifié 7 envoie ownerId:7 pour le modifier. Seul le propriétaire est autorisé. Que faire ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [71] [02] [17] [03] [05] [61] · Notices complètes à la fin ; liens cliquables.

SQL paramétré, ORM et transaction

Un ORM fournit des méthodes pour travailler avec la base ; ici Prisma est fixé à 6.19.0. Fragment à lire : db est un client PostgreSQL connecté et prisma un client généré pour Task, mappé sur task(id, title, done).

Données fournies : id est la clé primaire

id	title	done
1	Lire	false
2	Marcher	true

Deux lectures avec un résultat défini

```
const validatedId = 1;
const { rows } = await db.query(
  "SELECT id, title FROM task WHERE id = $1", [validatedId]
);
const todo = await prisma.task.findMany({
  where: {done:false}, orderBy: {id:"asc"},
  select: {title:true}
});
// rows : [{id:1, title:"Lire"}]
// todo : [{title:"Lire"}]
```

Garanties et limites

- \$1 est un paramètre : sa valeur ne devient pas du code SQL. Valider l'identifiant complète cette séparation ; concaténer une saisie permettrait une injection.
- findMany renvoie un tableau ; findUnique une ligne ou null. where filtre, select choisit les champs ; skip et take paginent, include ajoute une relation.
- PRIMARY KEY identifie ; FOREIGN KEY garantit la référence ; UNIQUE interdit un doublon ; NOT NULL impose une valeur. Les contraintes restent nécessaires malgré l'ORM.

Prisma et opérations groupées

Outil / notion	Effet
migrate dev	Créer et appliquer des migrations en développement.
migrate deploy	Appliquer les migrations existantes au déploiement.
generate	Produire le client ; ne pas créer les tables.
Transaction	Valider ensemble ou annuler ensemble les écritures.

ESSAIE SANS REGARDER LE CORRIGÉ · BAC-03

Une transaction renomme la tâche 1 puis échoue en supprimant la tâche 99, absente. Après l'annulation, quel titre garde la tâche 1 ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [22] [41] [60] [59] [58] · Notices complètes à la fin ; liens cliquables.

1 · Composants, props et état d'un formulaire

Un composant décrit une interface avec JSX, une syntaxe dans JavaScript. Une prop est une entrée du parent ; l'état est une valeur conservée dont la mise à jour déclenche un rendu.

Avant de lire le code

Mot	Définition
Rendu	Calcul de la description d'écran depuis les props et l'état.
useState	Valeur d'état et fonction de mise à jour.
Champ contrôlé	value lit l'état ; onChange le modifie.
Updater	Fonction calculant le nouvel état depuis le précédent.
key	Identité stable d'un élément rendu dans une liste.

Composant complet à placer dans App.jsx d'un environnement React

```
import { useState } from 'react';
function Name({ name, onName }) {
  return <input aria-label="Prénom" value={name}
    onChange={e => onName(e.target.value)} />;
}
export default function App() {
  const [name, setName] = useState('');
  const [count, setCount] = useState(0);
  const items = [{ id: 1, title: 'Lire' }, { id: 2, title: 'Marcher' }];
  return <main>
    <Name name={name} onName={setName} />
    <p>Bonjour {name || 'visiteur'} - {count}</p>
    <button onClick={() => setCount(n => n + 1)}>Ajouter</button>
    <ul>{items.map(item => <li key={item.id}>{item.title}</li>)}</ul>
  </main>;
}
```

Résultat et responsabilité

- Pour exécuter, créer un environnement React avec le guide officiel ; aucune API n'est nécessaire.
- Au départ : Bonjour visiteur - 0, avec Lire/Marcher. Saisir Lina puis cliquer deux fois donne Bonjour Lina - 2.
- App possède name : il transmet au champ sa valeur et la fonction permettant de la modifier.
- Pour changer un tableau d'état, produire une nouvelle valeur sans modifier l'ancienne.

ESSAIE SANS REGARDER LE CORRIGÉ · REA-01

Name possède-t-il l'état name ? Que fait la saisie sans onChange ? Pourquoi id convient-il à key ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [28] [33] [31] [29] [30] [27] · Notices complètes à la fin ; liens cliquables.

2 · Effets, promesses et paramètres d'URL

`useEffect` synchronise un composant avec un travail extérieur au rendu. Une Promise représente un résultat futur ; `await` attend ce résultat dans une fonction `async`.

Fragment à lire : Detail reçoit une prop id

```
import { useEffect, useState } from 'react';
async function load(id) {
  if (id !== 'lire') throw new Error('Activité inconnue'); return 'Lire';
}
function Detail({ id }) {
  const [state, set] = useState({ loading: true, text: '' });
  useEffect(() => {
    let active = true; set({ loading: true, text: '' });
    async function run() {
      try { const text = await load(id); if (active) set({loading:false,text}); }
      catch (e) { if (active) set({ loading: false, text: e.message }); }
    }
    run(); return () => { active = false; };
  }, [id]);
  return <p>{state.loading ? 'Chargement...' : state.text}</p>;
}
```

Dépendances et nettoyage

- `id` est une valeur réactive utilisée : `[id]` relance l'effet quand elle change.
- Le nettoyage empêche un ancien résultat d'actualiser un écran retiré ou une nouvelle sélection.
- `load` simule une source `async` : `lire` donne `Lire` ; tout autre `id` provoque `Activité inconnue`. `catch` traite cette erreur.
- `run` utilise `await` dans `try/catch`. L'effet retourne le nettoyage, pas une Promise.

Relier le composant à une URL

`BrowserRouter` fournit le contexte ; `Routes` regroupe les `Route`. Le chemin `/activites/:id` choisit un écran ; `useParams` lit `id`. Avec `/activites/lire`, l'écran transmet `id="lire"` à `Detail`. `Navigate` peut rediriger ; une navigation ne protège pas les données d'une API.

États attendus

Situation	Affichage
Chargement en cours	Chargement...
<code>id = lire</code> , succès	Lire
<code>id = autre</code> , échec	Activité inconnue

ESSAIE SANS REGARDER LE CORRIGÉ · REA-02

Pourquoi enlever `id` des dépendances est-il incorrect ? Que reçoit `useParams` pour `/activites/lire` ?

Correction dans la partie « Corrigés », à la fin du cours.

RÉFÉRENCES · [32] [35] [36] [34] · Notices complètes à la fin ; liens cliquables.

Corrigés des exercices

Compare ton raisonnement, puis reformule la correction avec tes propres mots. Le numéro renvoie à l'exercice de la leçon.

MER-01 · 1. Partir d'un besoin vérifiable (p. 3)

La demande est refusée et la date de retour reste inchangée. Le critère vérifie la limite d'un renouvellement.

MER-02 · 2. Définir les données et lire les cardinalités (p. 4)

Le livre sans copie est autorisé par 0,n. L'exemplaire sans livre est interdit par 1,1. La cardinalité proche d'une entité se lit depuis une occurrence de cette entité.

MER-03 · 3. Préserver les liens : binaire ou ternaire ? (p. 5)

Non. Bob-JavaScript n'existe pas parmi les paires enseignant-matière. À l'inverse, les trois paires d'Alice-SQL-C2 existent bien, alors que le triplet complet est absent.

MER-04 · 4. Du MCD au MLD : 1:n et n:n (p. 6)

Les deux lignes portent livre_id = 7. Les clés primaires exemplaire_id restent distinctes : 10 et 11.

MER-05 · 5. Liens 1:1, ternaire et MPD PostgreSQL (p. 7)

Le dossier 11 est refusé : personne_id = 1 existe déjà et doit être unique. Le dossier 12 est refusé : la personne 3 n'existe pas. La personne 2 peut rester sans dossier.

MER-06 · 6. Dépendances, clés et première forme normale (p. 8)

Non. La règle autorise des homonymes. L'unicité observée dans cet échantillon ne garantit pas celle de tous les états valides.

MER-07 · 7. Normaliser : 2NF, 3NF et BCNF (p. 9)

Non. La dépendance reste vraie et le couple commande-produit reste une clé candidate si sa règle d'unicité est conservée. Il faut examiner toutes les clés candidates.

MER-08 · 8. Cas d'utilisation UML : include et extend (p. 10)

Retirer inclut Imprimer le reçu : la flèche include va de Retirer vers Imprimer le reçu. Le comportement appartient désormais à chaque retrait.

TYP-01 · 1 · Des types pour relire le code avant son exécution (p. 11)

Le tuple contient exactement deux positions typées : texte puis nombre. Non : les annotations ne contrôlent pas le JSON pendant l'exécution ; il faut vérifier ses champs et valeurs.

Corrigés · suite

TYP-02 · 2 · Accepter une valeur inconnue, puis vérifier (p. 12)

{ status: 'ok', data: 'Alice' }. La variante ok possède data, pas message ; le discriminant a précisément sélectionné cette forme.

TYP-03 · 3 · Réutiliser un type et attendre un calcul (p. 13)

undefined ; Promise<number> ; non. Partial change une description statique, sans ajouter de validation à l'exécution.

SQL-01 · 1 · Tables, identifiants et intégrité (p. 14)

Non : le client 3 n'existe pas, donc la clé étrangère bloque la première. Le total -1 viole CHECK(total >= 0), donc la seconde est refusée.

SQL-02 · 2 · Lire, filtrer, trier et découper (p. 15)

La commande 10 avec total 30. La commande 11 vaut 20 et ne passe pas le filtre.

SQL-03 · 3 · Joindre, comprendre NULL et compter (p. 16)

Seule Alice reste, avec 2 commandes et total 50. COUNT(*) donnerait 1 pour Bilal à cause de la ligne conservée par LEFT JOIN.

SQL-04 · 4 · Existence, modifications et travail fiable (p. 17)

30 : ROLLBACK annule 99. Sans WHERE, UPDATE viserait les deux commandes de cet exemple.

DOC-01 · De l'image au conteneur (p. 18)

Non. EXPOSE documente le port interne. La publication vient de -p : 8080 sur la machine dirige vers 3000 dans le conteneur.

DOC-02 · Ports, réseau Compose et données persistantes (p. 20)

api utilise toujours db:5432 sur le réseau Compose. L'outil sur la machine perd cet accès publié : localhost:15432 n'atteint plus db.

TES-01 · Une règle, des limites et une assertion (p. 22)

1, 2, 8 et 9. Les attendus sont false, true, true, false. Ajouter d'autres classes comme une chaîne et un nombre décimal.

TES-02 · Attendre, tester HTTP et nettoyer (p. 23)

404 et {error:"Absente"}. Ce test d'intégration vérifie l'échange entre serveur et client HTTP ; aucune base externe n'intervient.

Corrigés · suite

BAC-01 · La requête traverse des responsabilités (p. 25)

Le contrôleur convertit et valide l'identifiant ; le service demande au repository la tâche. Un middleware qui ne répond pas et n'appelle pas next() laisse la requête en attente.

BAC-02 · Valider les données, vérifier les droits (p. 26)

Refuser avec 403, sans modification : l'identité 7 diffère du propriétaire 9 enregistré. Le corps ne crée aucun droit.

BAC-03 · SQL paramétré, ORM et transaction (p. 27)

Lire : aucune écriture du groupe n'est conservée. Une transaction ne remplace pas une isolation adaptée aux accès concurrents ni les contraintes métier.

REA-01 · 1 · Composants, props et état d'un formulaire (p. 28)

Non : App possède l'état et Name reçoit ses props. Sans mise à jour, le champ contrôlé ne suit pas la saisie. id distingue durablement les deux activités.

REA-02 · 2 · Effets, promesses et paramètres d'URL (p. 29)

L'effet lit id et doit suivre sa nouvelle valeur ; [] garderait une synchronisation périmée. useParams donne ici id égal à "lire".

Lire les références

Les situations sont créées pour apprendre. Les liens ci-dessous donnent les références des notions : documentation des outils, spécification UML ou travaux académiques selon le sujet.

[01] Agile Alliance · Modèle de user story

Documentation officielle

<https://agilealliance.org/glossary/user-story-template/>

[02] OWASP · vérifier les permissions

Documentation officielle

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

[03] OWASP · stockage des mots de passe

Documentation officielle

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[04] Sibel Adali, RPI · Dépendances, formes normales et décomposition

Référence académique

https://cs.rpi.edu/~sibel/csci4380/fall2020/course_notes/normalization.html

[05] MDN · propriétés d'un cookie HTTP

Documentation officielle

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie>

[06] Docker Compose · réseau et noms de services

Documentation officielle

<https://docs.docker.com/compose/how-tos/networking/>

[07] Docker Compose · démarrage et disponibilité

Documentation officielle

<https://docs.docker.com/compose/how-tos/startup-order/>

[08] Docker · volumes persistants

Documentation officielle

<https://docs.docker.com/engine/storage/volumes/>

[09] Docker · réutilisation du cache

Documentation officielle

<https://docs.docker.com/get-started/docker-concepts/building-images/using-the-build-cache/>

[10] Docker · définition d'un conteneur

Documentation officielle

<https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>

Références · suite

[11] Docker · définition d'une image

Documentation officielle

<https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/>

[12] Docker Compose · arrêt et suppression

Documentation officielle

<https://docs.docker.com/reference/cli/docker/compose/down/>

[13] Docker · instructions du Dockerfile

Documentation officielle

<https://docs.docker.com/reference/dockerfile/>

[14] Express 5 · traitement des erreurs

Documentation officielle

<https://expressjs.com/en/guide/error-handling/>

[15] Express 5 · chaîne de middlewares

Documentation officielle

<https://expressjs.com/en/guide/using-middleware/>

[16] Express 5 · premier serveur

Documentation officielle

<https://expressjs.com/en/starter/hello-world/>

[17] jsonwebtoken · signature, verify et decode

Documentation officielle

<https://github.com/auth0/node-jwebtoken>

[18] Mocodo · Manuel de référence de l'auteur

Documentation officielle

https://laowantong.github.io/mocodo/doc/fr_refman.html

[19] Microsoft Learn · cycle Red, Green, Refactor

Documentation officielle

<https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions-1/contact-manager/iteration-6-use-test-driven-development-cs>

[20] Microsoft Learn · organiser des tests unitaires avec Arrange, Act, Assert

Documentation officielle

<https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-best-practices>

[21] Pierre Gérard · Modélisation des systèmes d'information, cours universitaire

Référence académique

<https://lipn.univ-paris13.fr/~gerard/docs/cours/merise-slides.pdf>

Références · suite

[22] node-postgres · Requêtes paramétrées

Documentation officielle

<https://node-postgres.com/features/queries>

[23] Node.js · vérifier un résultat avec assert

Documentation officielle

<https://nodejs.org/api/assert.html>

[24] Node.js · créer un serveur HTTP

Documentation officielle

<https://nodejs.org/api/http.html>

[25] Node.js · exécuter des tests

Documentation officielle

<https://nodejs.org/api/test.html>

[26] Nanci et Espinasse · Merise, deuxième génération (ouvrage des auteurs)

Référence académique

<https://pageperso.lis-lab.fr/bernard.espinasse/wp-content/uploads/2021/12/LivreMerisePDF-total-12sept14.pdf>

[27] React · Créer une application

Documentation officielle

<https://react.dev/learn/build-a-react-app-from-scratch>

[28] React · Transmettre des props

Documentation officielle

<https://react.dev/learn/passing-props-to-a-component>

[29] React · Partager un état

Documentation officielle

<https://react.dev/learn/sharing-state-between-components>

[30] React · Mettre à jour un tableau

Documentation officielle

<https://react.dev/learn/updating-arrays-in-state>

[31] React · Champs input

Documentation officielle

<https://react.dev/reference/react-dom/components/input>

[32] React · useEffect et nettoyage

Documentation officielle

<https://react.dev/reference/react/useEffect>

Références · suite

[33] React · useState

Documentation officielle

<https://react.dev/reference/react/useState>

[34] React Router · Navigate

Documentation officielle

<https://reactrouter.com/api/components/Navigate>

[35] React Router · BrowserRouter

Documentation officielle

<https://reactrouter.com/api/declarative-routers/BrowserRouter>

[36] React Router · useParams

Documentation officielle

<https://reactrouter.com/api/hooks/useParams>

[37] Stéphane Crozat · Théorie de la normalisation relationnelle

Référence académique

<https://stph.scenari-community.org/bdd/nor1.pdf>

[38] tsx · Exécution et vérification des types

Documentation officielle

<https://tsx.is/typescript>

[39] Vitest · installer et lancer des tests

Documentation officielle

<https://vitest.dev/guide/>

[40] OMG · UML 2.5.1, standard et document normatif, chapitre 18

Norme / spécification

<https://www.omg.org/spec/UML/2.5.1>

[41] PostgreSQL · Contraintes

Documentation officielle

<https://www.postgresql.org/docs/current/ddl-constraints.html>

[42] PostgreSQL · RETURNING

Documentation officielle

<https://www.postgresql.org/docs/current/dml-returning.html>

[43] PostgreSQL · Comparaisons et NULL

Documentation officielle

<https://www.postgresql.org/docs/current/functions-comparison.html>

Références · suite

[44] PostgreSQL · COALESCE et expressions conditionnelles

Documentation officielle

<https://www.postgresql.org/docs/current/functions-conditional.html>

[45] PostgreSQL · Sous-requêtes

Documentation officielle

<https://www.postgresql.org/docs/current/functions-subquery.html>

[46] PostgreSQL · Index

Documentation officielle

<https://www.postgresql.org/docs/current/indexes.html>

[47] PostgreSQL · LIMIT et OFFSET

Documentation officielle

<https://www.postgresql.org/docs/current/queries-limit.html>

[48] PostgreSQL · ORDER BY

Documentation officielle

<https://www.postgresql.org/docs/current/queries-order.html>

[49] PostgreSQL · PREPARE et paramètres

Documentation officielle

<https://www.postgresql.org/docs/current/sql-prepare.html>

[50] PostgreSQL · Agrégats

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-agg.html>

[51] PostgreSQL · DELETE

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-delete.html>

[52] PostgreSQL · Jointures

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-join.html>

[53] PostgreSQL · INSERT et données d'exemple

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-populate.html>

[54] PostgreSQL · Interroger une table

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-select.html>

Références · suite

[55] PostgreSQL · Transactions

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-transactions.html>

[56] PostgreSQL · UPDATE

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-update.html>

[57] PostgreSQL · EXPLAIN

Documentation officielle

<https://www.postgresql.org/docs/current/using-explain.html>

[58] Prisma ORM 6 · transactions

Documentation officielle

<https://www.prisma.io/docs/orm/v6/prisma-client/queries/transactions>

[59] Prisma ORM 6 · migrations en développement et déploiement

Documentation officielle

<https://www.prisma.io/docs/orm/v6/prisma-migrate/workflows/development-and-production>

[60] Prisma ORM 6 · méthodes du client

Documentation officielle

<https://www.prisma.io/docs/orm/v6/reference/prisma-client-reference>

[61] RFC 9110 · codes de réponse HTTP

Norme / spécification

<https://www.rfc-editor.org/rfc/rfc9110.html#name-status-codes>

[62] TypeScript · Bases et compilateur

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/basic-types.html>

[63] TypeScript · Types courants

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

[64] TypeScript · Fonctions

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/functions.html>

[65] TypeScript · Génériques

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/generics.html>

Références · suite

[66] TypeScript · Modules et imports de types

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/modules.html>

[67] TypeScript · Narrowing et unions discriminées

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html>

[68] TypeScript · Types d'objets et tuples

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/objects.html>

[69] TypeScript · Types utilitaires

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/utility-types.html>

[70] TypeScript · Référence tsconfig

Documentation officielle

<https://www.typescriptlang.org/tsconfig/>

[71] Zod · validation des données à l'exécution

Documentation officielle

<https://zod.dev/basics>